Fresher Training Task – Chrome DevTools & Performance Analysis

Duration: 1 Day | Audience: QA/Dev Freshers

Training Objective

Understand Chrome DevTools for debugging, network analysis, browser storage, accessibility and performance testing including API performance investigation.

Learning Outcomes

- Inspect and navigate the DOM using Elements panel.
- Understand Styles vs Computed and CSS inheritance.
- Inspect Event Listeners and Accessibility Tree.
- Use Console and Sources for debugging with breakpoints and call stack.
- Analyze Network requests, waterfall and throttling.
- Perform basic frontend and API performance analysis.
- Capture Performance and Memory profiles.
- Inspect Cookies, Cache, Local Storage and Session Storage.
- Generate Lighthouse reports.

Practice Websites

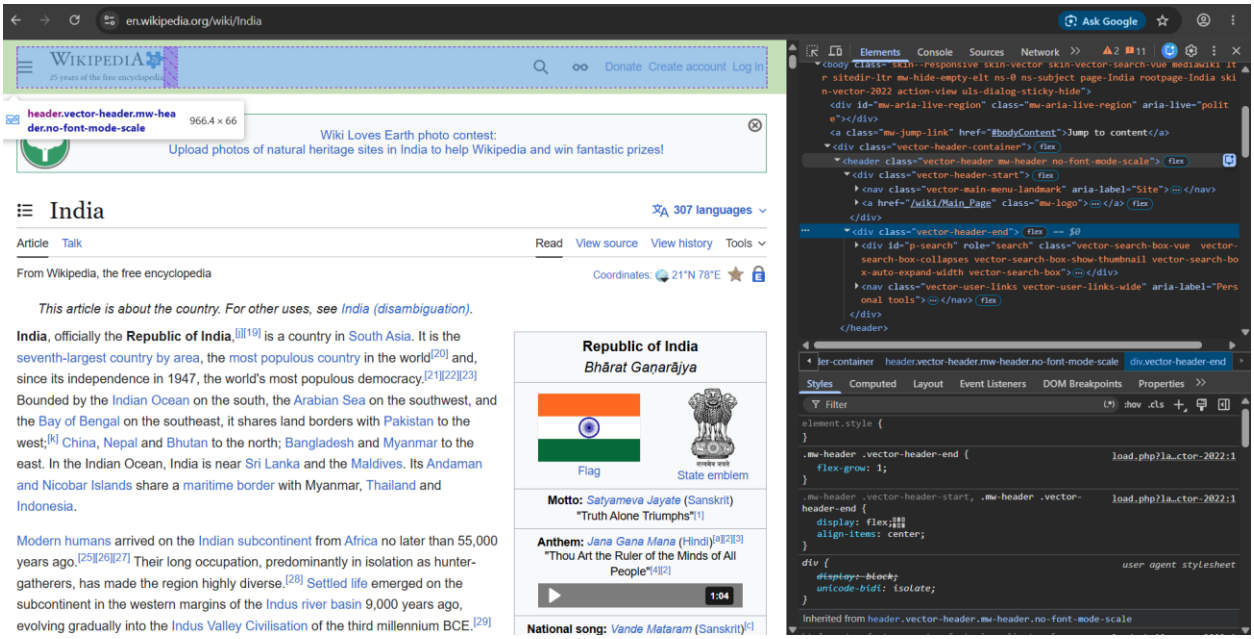
- https://demoqa.com
- https://www.wikipedia.org
- https://developer.chrome.com/docs/devtools/
- https://www.saucedemo.com/
- https://www.browserstack.com/

Tasks (1 Day)

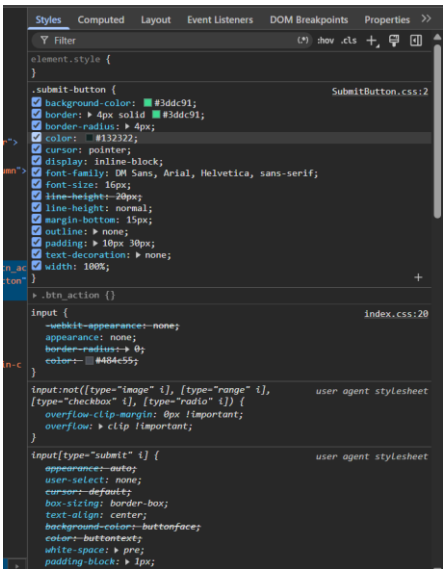
Task 1 – Elements & Accessibility

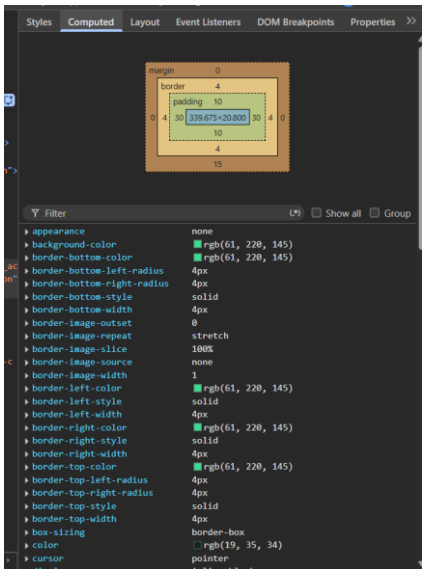
Inspect DOM,Identify parent/child relationships

- Navigation tab is parent and its child elements logo, search and nav links



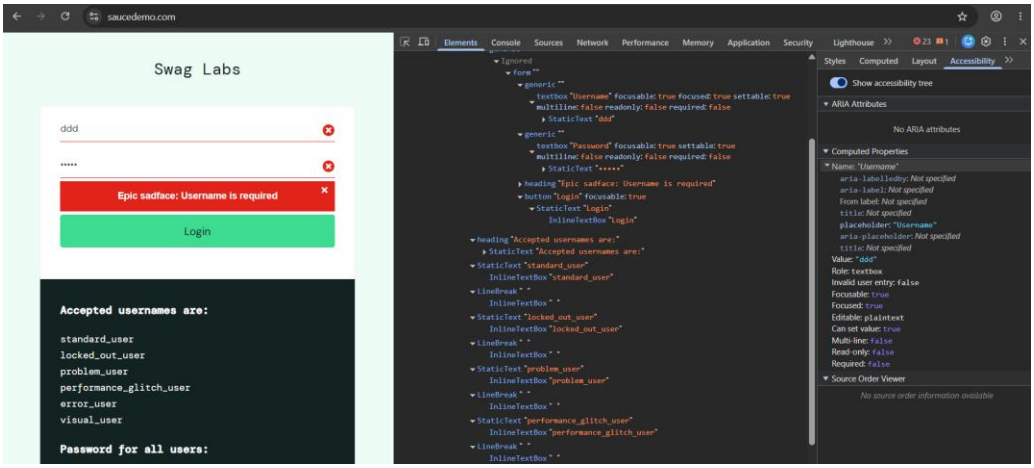
Compare Styles vs Computed – Styles shows CSS properties taken from root parent to leaf child, also showing some properties being overriden, Computed shows final browser rendered Property





Inspect Accessibility Tree for 3 elements.

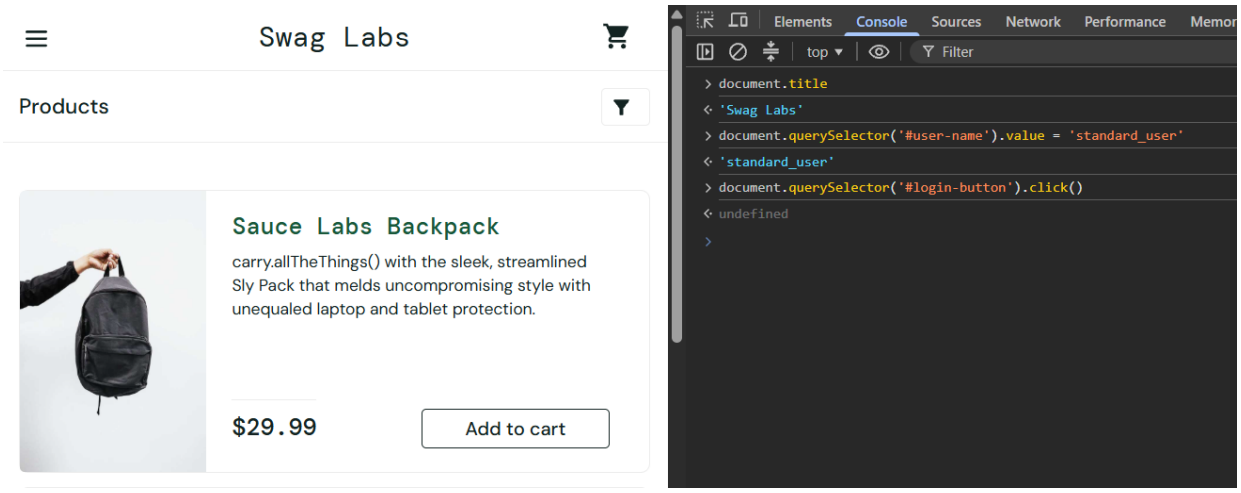
1. Username
2. Password
3. Login



Task 2 – Console & Sources

1. Use console to query DOM and perform some action

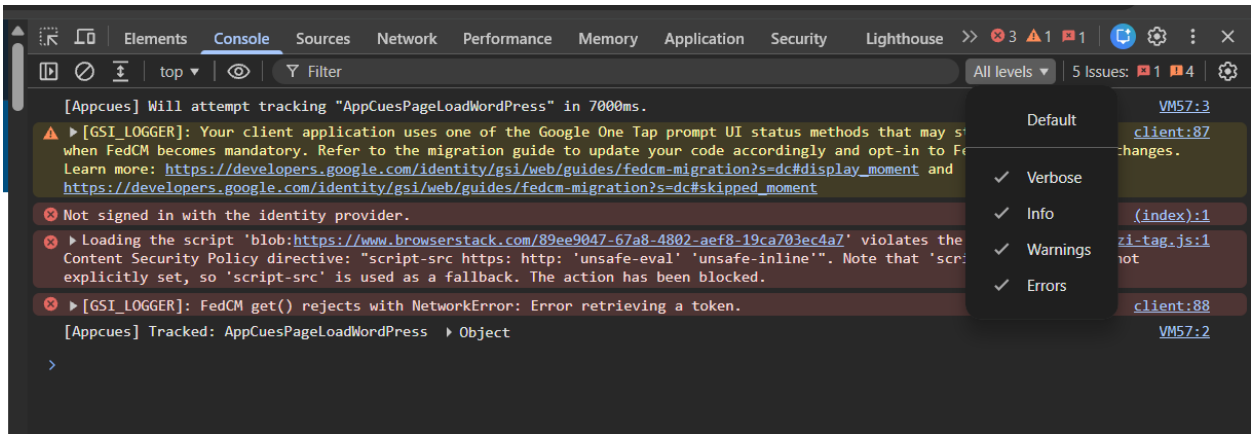
Using query selector filled the form and performed a click event



2. Identify different types of console messages

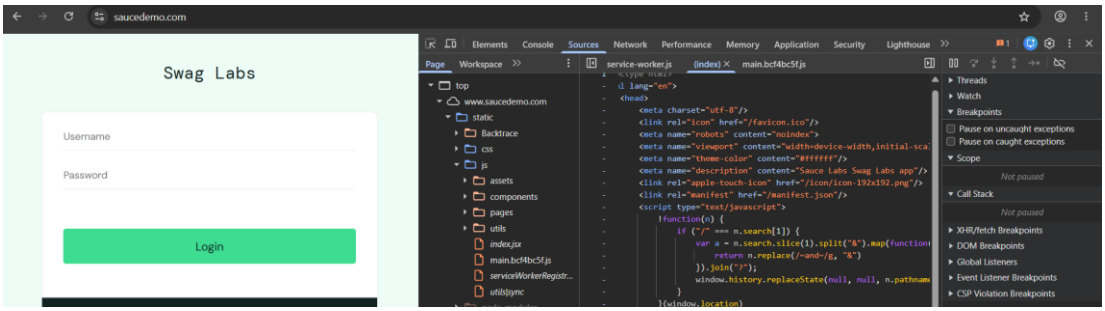
- There are 4 types of console messages

a. Verbose
b. Information
c. Warnings
d. Errors



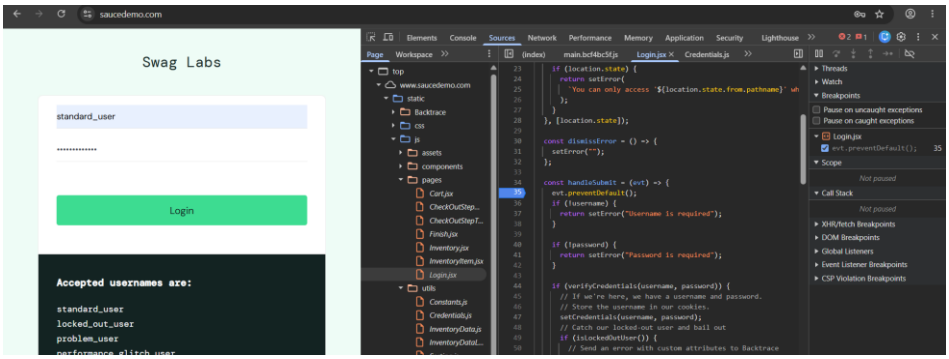
3. In Sources tab- Locate the JavaScript file used by the page.

The main index file of saucedemo.com is using /static/js/main.bcf4bc5f.js

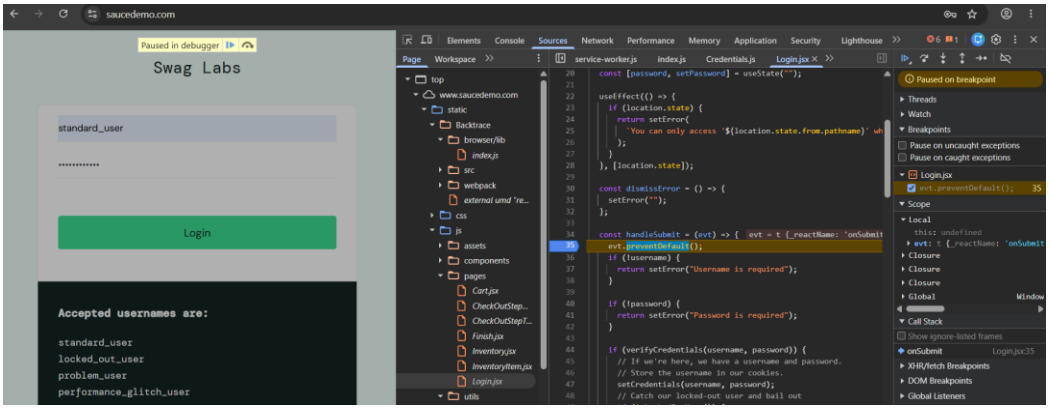


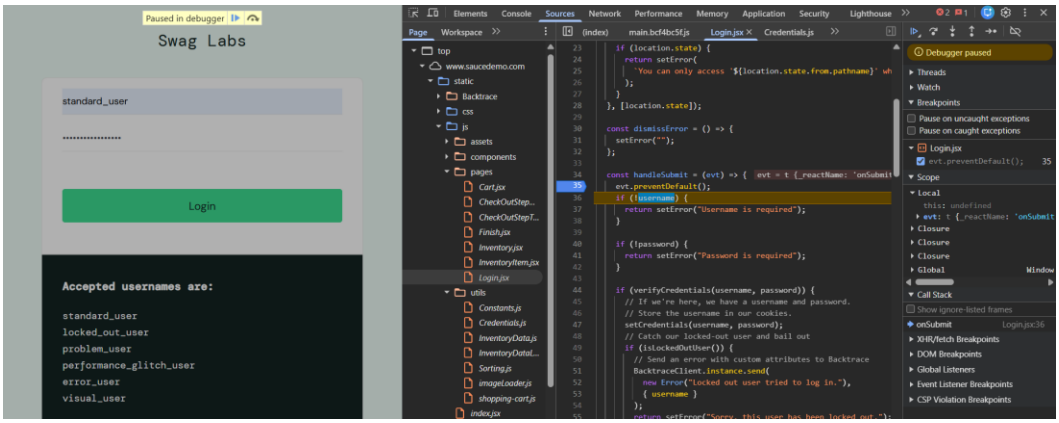
4. Set a breakpoint on a line that executes when a button is clicked.

- Breakpoint set on handleSubmit on line 35

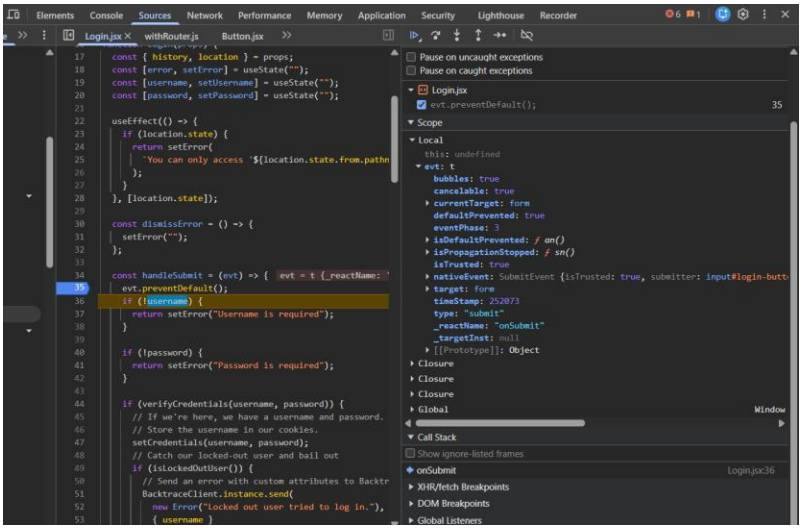


5. Click the button and observe that execution pauses.





6. Identify:
    a. Current execution line - 35 is starting, then highlight the line currently executing
    b. Local variables – Displays the local variable in function eg. username, password
    c. Call Stack – Sequence of Function calls - onSubmit, verifyCredentials ,setCredentials ,isProblemUser, (anonymous)

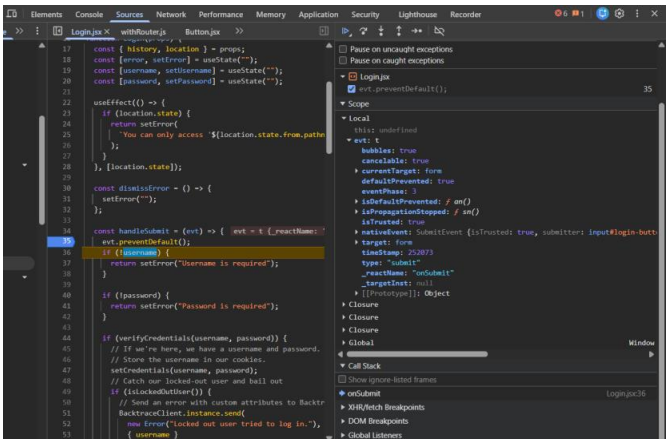


7. Use the following debugging controls:
    a. Resume (Continue)

b. Step Over – Next Line of code without going in function calls
c. Step Into – step into next function call in current line
d. Step Out – step of current function call going back to the call

8. Analyze the Callstack and Variables panel

Tracks data state and execution flow to identify logic errors



9. Record / document your findings

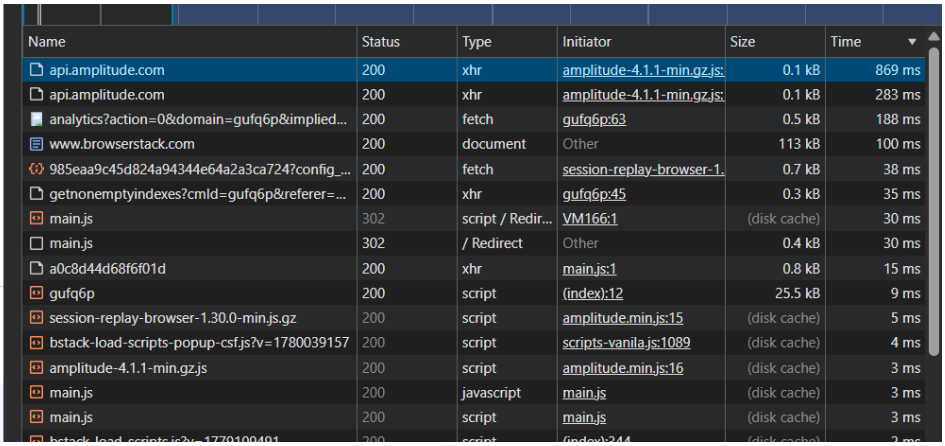
All the observations are noted and attached above

Task 3 – Network Performance Analysis

Identify HTML/CSS/JS/API requests, inspect headers and payloads, use filters, preserve logs and throttling.

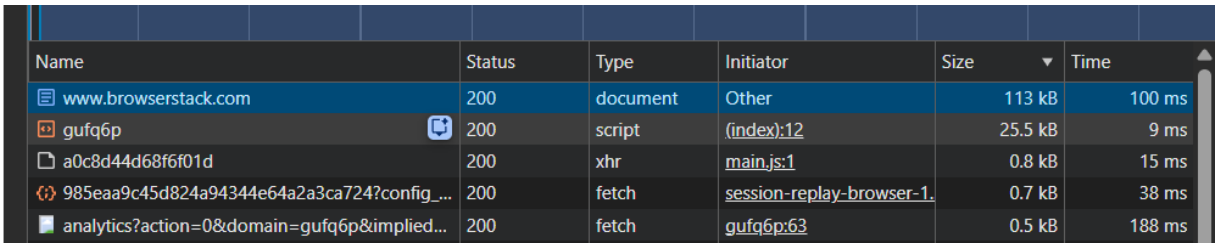
Analyzing on BrowserStack.com

1. Sort requests by Time and identify:
    a. Slowest request - https://api.amplitude.com/ - 869ms



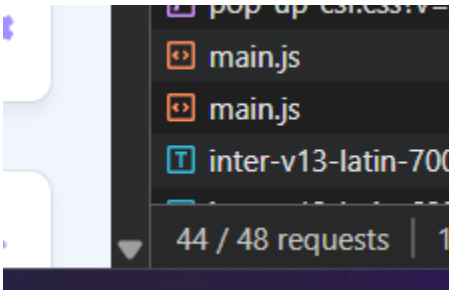
Name	Status	Type	Initiator	Size	Time
api.amplitude.com	200	xhr	amplitude-4.1.1-min.gz.js:	0.1 kB	869 ms
api.amplitude.com	200	xhr	amplitude-4.1.1-min.gz.js:	0.1 kB	283 ms
analytics?action=0&domain=gufq6p&implied...	200	fetch	gufq6p:63	0.5 kB	188 ms
www.browserstack.com	200	document	Other	113 kB	100 ms
985eaa9c45d824a94344e64a2a3ca724?config_...	200	fetch	session-replay-browser-1.	0.7 kB	38 ms
getnonemptyindexes?cmId=gufq6p&referer=...	200	xhr	gufq6p:45	0.3 kB	35 ms
main.js	302	script / Redir...	VM166:1	(disk cache)	30 ms
main.js	302	/ Redirect	Other	0.4 kB	30 ms
a0c8d44d68f6f01d	200	xhr	main.js:1	0.8 kB	15 ms
gufq6p	200	script	(index):12	25.5 kB	9 ms
session-replay-browser-1.30.0-min.js.gz	200	script	amplitude.min.js:15	(disk cache)	5 ms
bstack-load-scripts-popup-csf.js?v=1780039157	200	script	scripts-vanila.js:1089	(disk cache)	4 ms
amplitude-4.1.1-min.gz.js	200	script	amplitude.min.js:16	(disk cache)	3 ms
main.js	200	javascript	main.js	(disk cache)	3 ms
main.js	200	script	main.js	(disk cache)	3 ms

    b. Largest request - www.browserstack.com - size 113kb

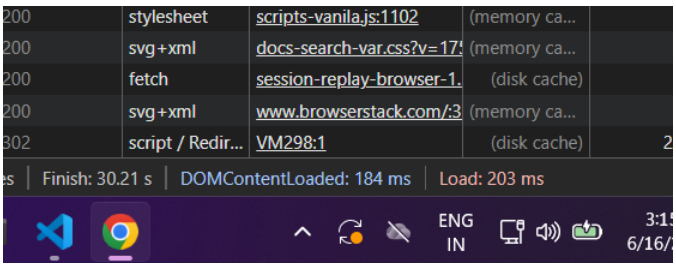


Name	Status	Type	Initiator	Size	Time
www.browserstack.com	200	document	Other	113 kB	100 ms
gufq6p	200	script	(index):12	25.5 kB	9 ms
a0c8d44d68f6f01d	200	xhr	main.js:1	0.8 kB	15 ms
985eaa9c45d824a94344e64a2a3ca724?config_...	200	fetch	session-replay-browser-1.	0.7 kB	38 ms
analytics?action=0&domain=gufq6p&implied...	200	fetch	gufq6p:63	0.5 kB	188 ms

c. Total number of requests - 48

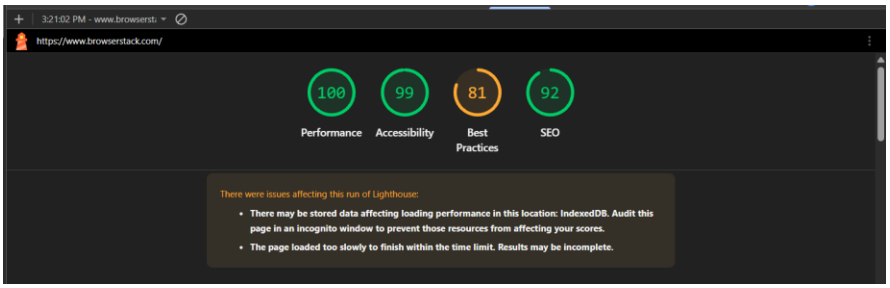


d. Total page load time - 203ms



Task 4 – Audit

1. Generate a basic Lighthouse report and identify improvement opportunities.
   a. Select: Performance A11y and Best Practices
   b. Record

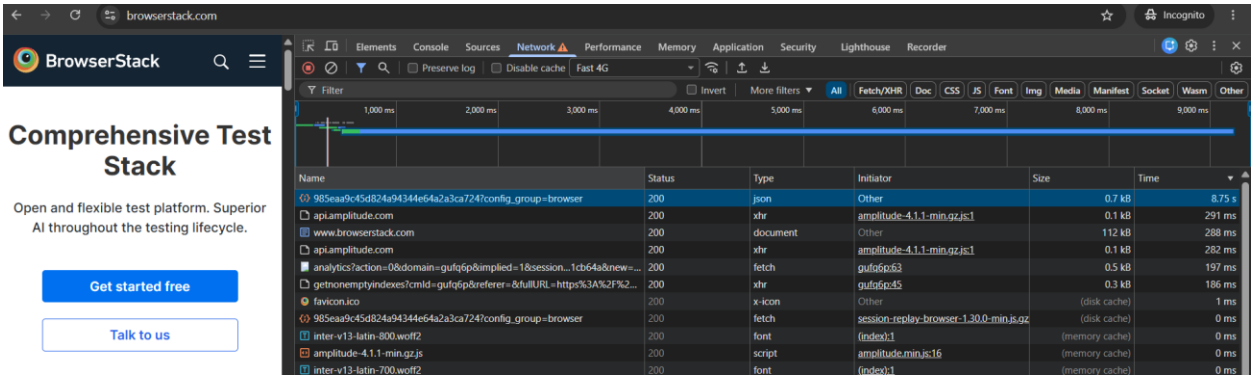


c. List few important recommendations shown by lighthouse.

- Uses deprecated APIs 3 warnings found
- Use efficient cache lifetimes Est savings of 29 KiB - A long cache lifetime can speed up repeat visits to your page.

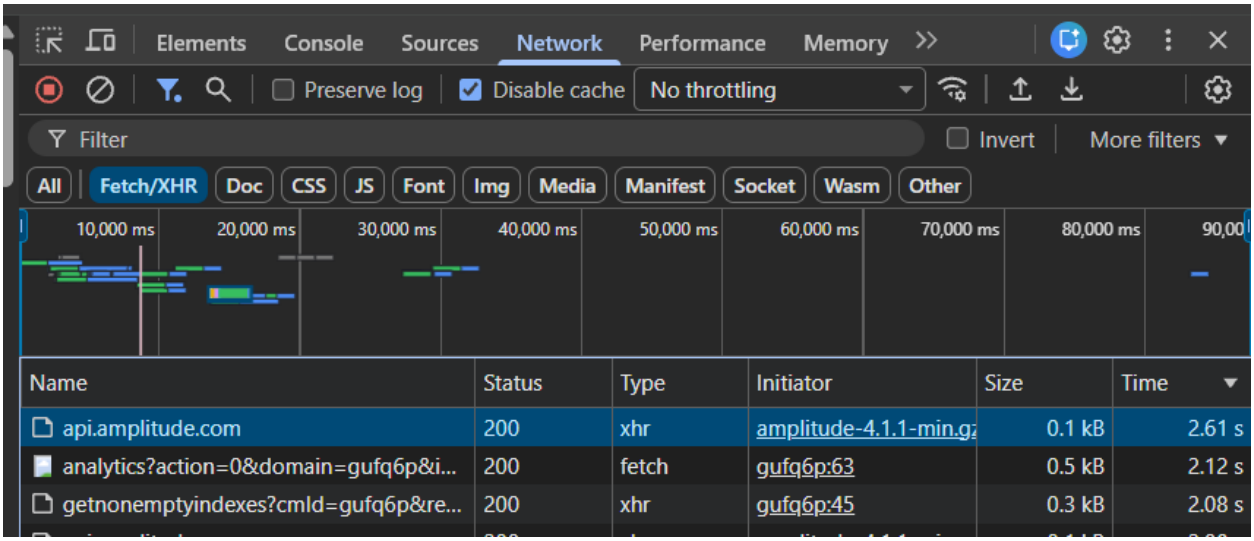
- Legacy JavaScript Est savings of 19 KiB - Polyfills and transforms enable older browsers to use new JavaScript features. However, many aren't necessary for modern browsers. Consider modifying your JavaScript build process to not transpile Baseline features, unless you know you must support older browsers
- LCP Breakdown - Each subpart has specific improvement strategies. Ideally, most of the LCP time should be spent on loading the resources, not within delay

2. Analyze a webpage for:
    a. Slow page investigation Large file bundles delayed Page rendering

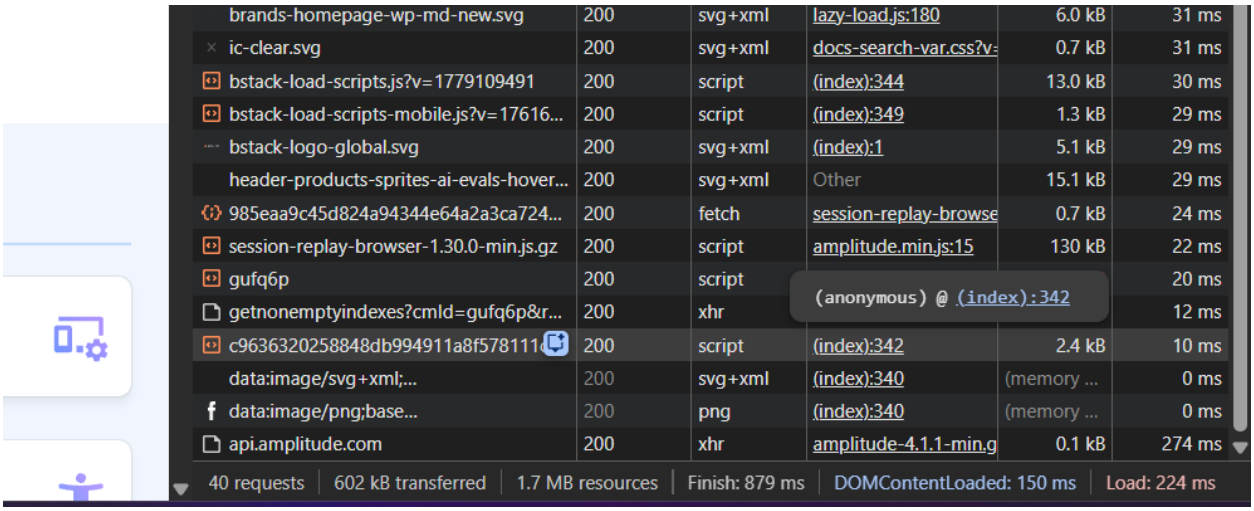


    b. Slow API investigation

API response took longer than static files



    c. Page load timing



Task 5 – API Performance

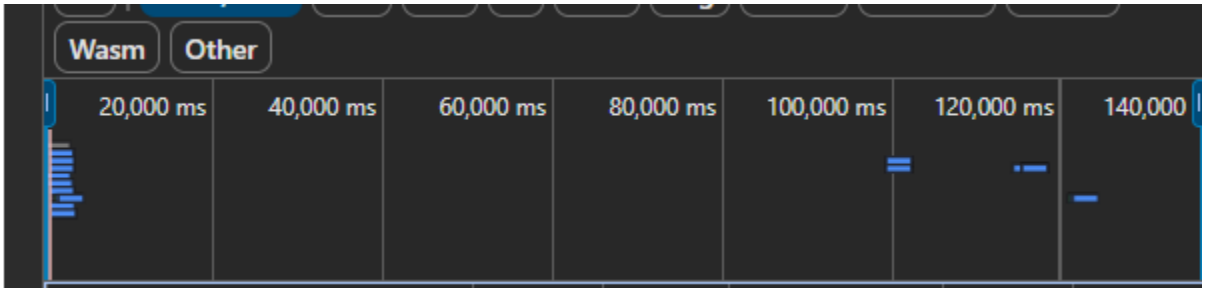
Select 3 API calls and record url, response time, size, status, TTFB and waterfall timing. Repeat under Slow 4G throttling report comparision.

	Record URL	Response Time	Size	Status	TTFB
Normal	https://eds.browserstack.com/send_event	206ms	0.6kb	200	204.61ms
Slow 4G		599ms	0.6kb	200	585.35ms
Normal	https://api.amplitude.com/	266ms	0.1kb	200	265.22ms
Slow 4G		620ms	0.1kb	200	582.44ms

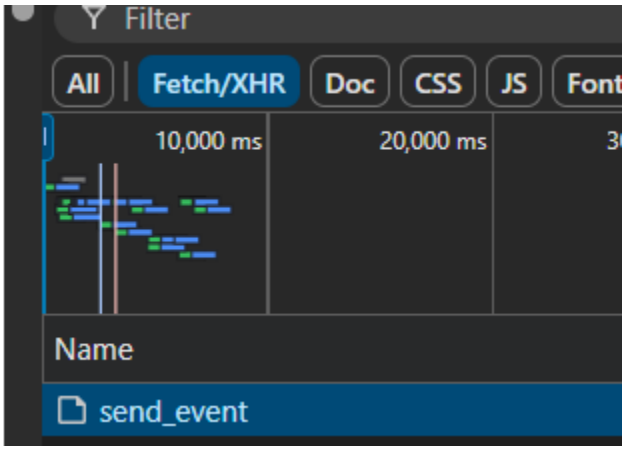
Normal	https://consent.trustarc.com/v2/consentcategories/getnonemptyindexes?cmId=gufq6p&referer=&fullURL=https%3A%2F%2Fwww.browserstack.com%2F&category=	35 ms	0.3 kb	200	6.75 ms
Slow 4G		613 ms	0.3 kb	200	591.18 ms

- Waterfall timing

Normal

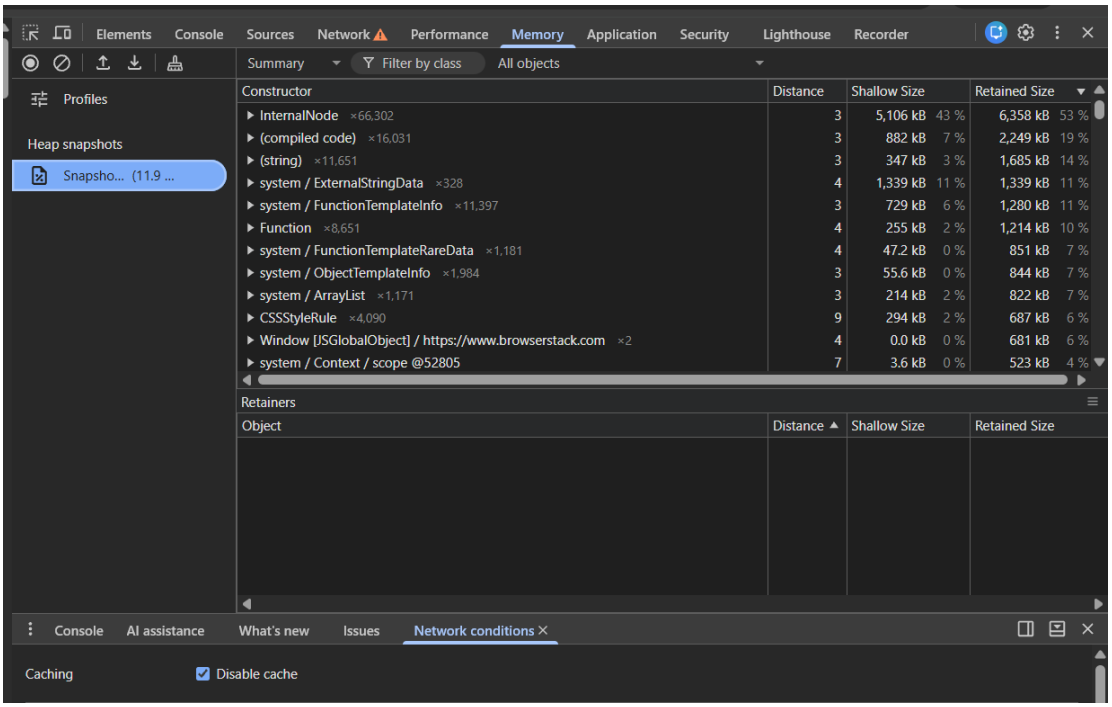


With Slow 4G throttling

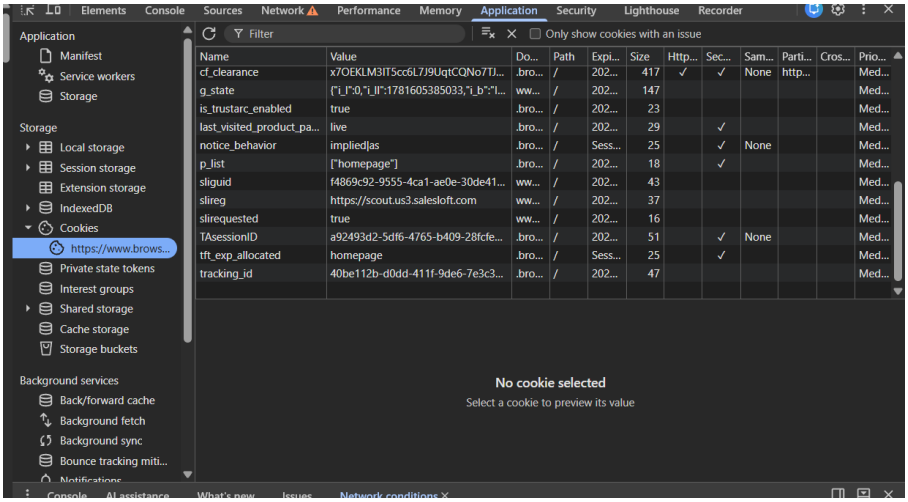


Task 6 – Memory & Storage

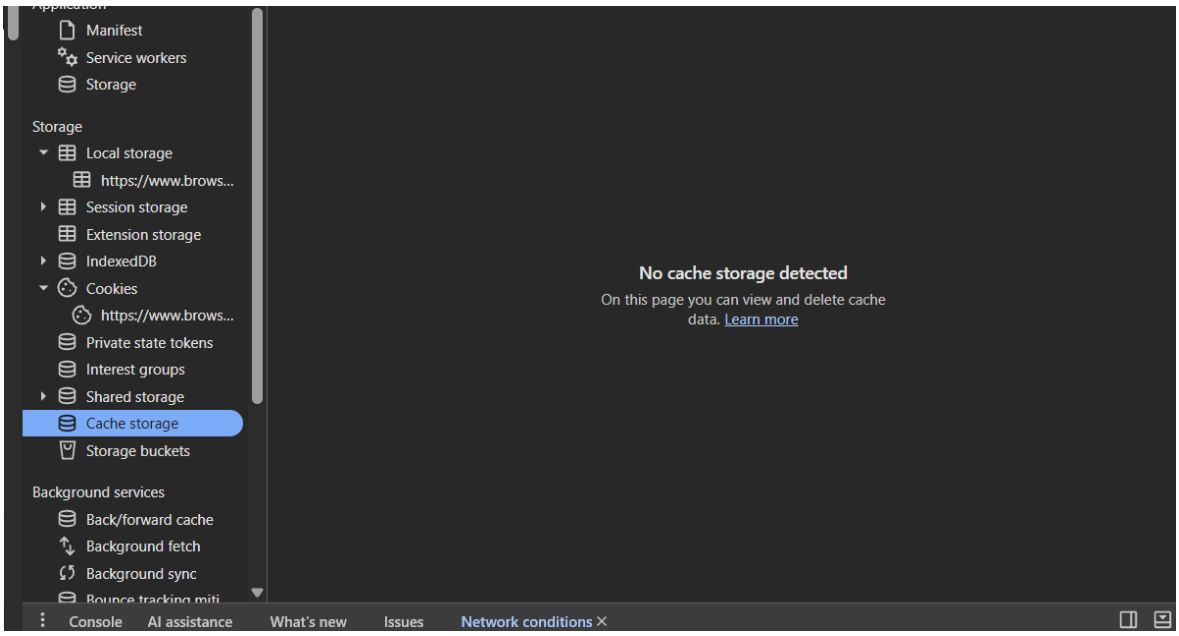
Capture Heap Snapshot



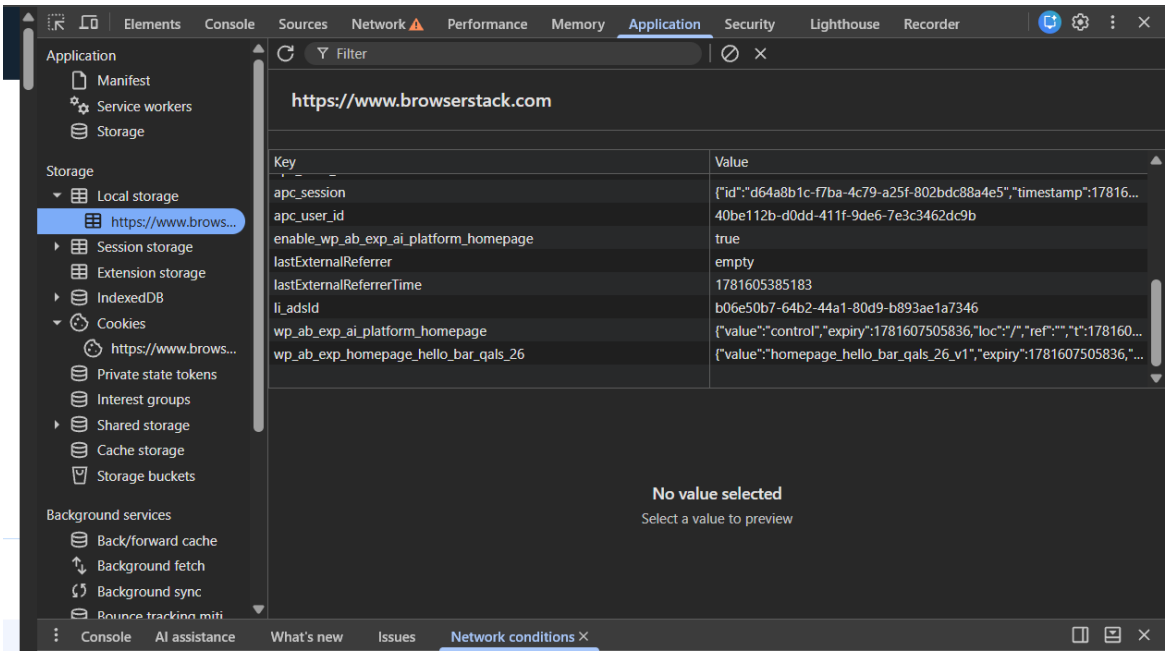
Inspect Cookies



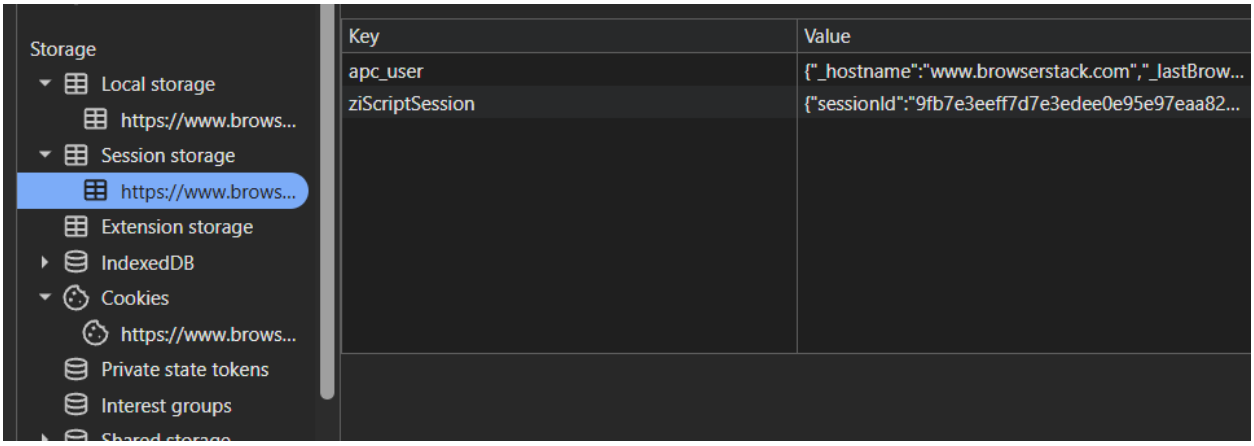
Cache



Local Storage



Session Storage



Document differences

- **Heap Snapshot**: Analyzes active JavaScript memory allocation and objects in real-time, unlike the others which are used to save data.
- **Cookies**: Automatically sent to the server with every HTTP request, whereas the other storage APIs remain strictly on the client.

- **Cache Storage**: Stores structured HTTP request/response pairs primarily for Service Workers and offline functionality, instead of raw strings.
- **Local Storage**: Data persists indefinitely across browser restarts until explicitly cleared, making it ideal for long-term client-side settings.
- **Session Storage**: Data expires immediately when the browser tab/window is closed, whereas Local Storage survives

Required Deliverables

- Document with screenshots for each task.
- Lighthouse report with observations.
- Performance profile screenshot and analysis.
- API performance comparison table (Normal vs Slow 4G).
- Storage comparison (Cookies vs Local vs Session vs Cache).
- Summary of at least 5 performance improvement recommendations.

Evaluation Criteria

Area	Weightage
Elements & Debugging	30%
Network Analysis	25%
API Performance Analysis	20%
Performance & Lighthouse	15%
Memory & Storage	10%